ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ _____

ПРЕПОДАВАТЕЛЬ

старший преподаватель		Т.А. Суетина
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

Процедурная генерация лабиринтов

по курсу: Мультимедийный практикум

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4329		Д.С. Шаповалова
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1. Цель работы:

Изучить базовые принципы процедурной генерации уровней на примере создания игры про бег в лабиринте. Освоить генерацию данных лабиринтов, их использование для построения меша.

2. Задание:

1. Задать свой размер лабиринта.
2. Выполнить классификацию полученного алгоритма по измерению, гиперразмерности, топологии, тесселяции, маршрутизации, текстуре и фокусу.

3. Ход работы:

3.1. Настройка архитектуры проекта и базовой структуры кода

Была создана базовая архитектура проекта:

Добавлен пустой GameObject Controller в сцену

Разработаны скрипты GameController и MazeConstructor

Настроена связь между компонентами с помощью атрибута [RequireComponent]

Реализовано свойство data для хранения двумерного массива данных лабиринта (1 – стена, 0 – пустое пространство)

Добавлен метод OnGUI() для отладочного отображения данных лабиринта в текстовом виде

Инициализированы базовые значения по умолчанию (массив 3×3 со стенами по периметру)

3.2. Разработка алгоритма генерации данных лабиринта

Был создан и реализован генератор данных:

Разработан класс MazeDataGenerator (не наследуется от MonoBehaviour)

Реализован метод FromDimensions(), принимающий размеры сетки (количество строк и столбцов)

Внедрён алгоритм генерации, который:

Устанавливает стены по границам массива

Проходит через каждую вторую ячейку сетки

С вероятностью placementThreshold размещает стены и блокирует соседние ячейки

Использует нечётные числа для размеров для корректного окружения стенами

Добавлена проверка на использование нечётных чисел с выводом предупреждения в консоль

Настроена интеграция генератора данных в MazeConstructor.

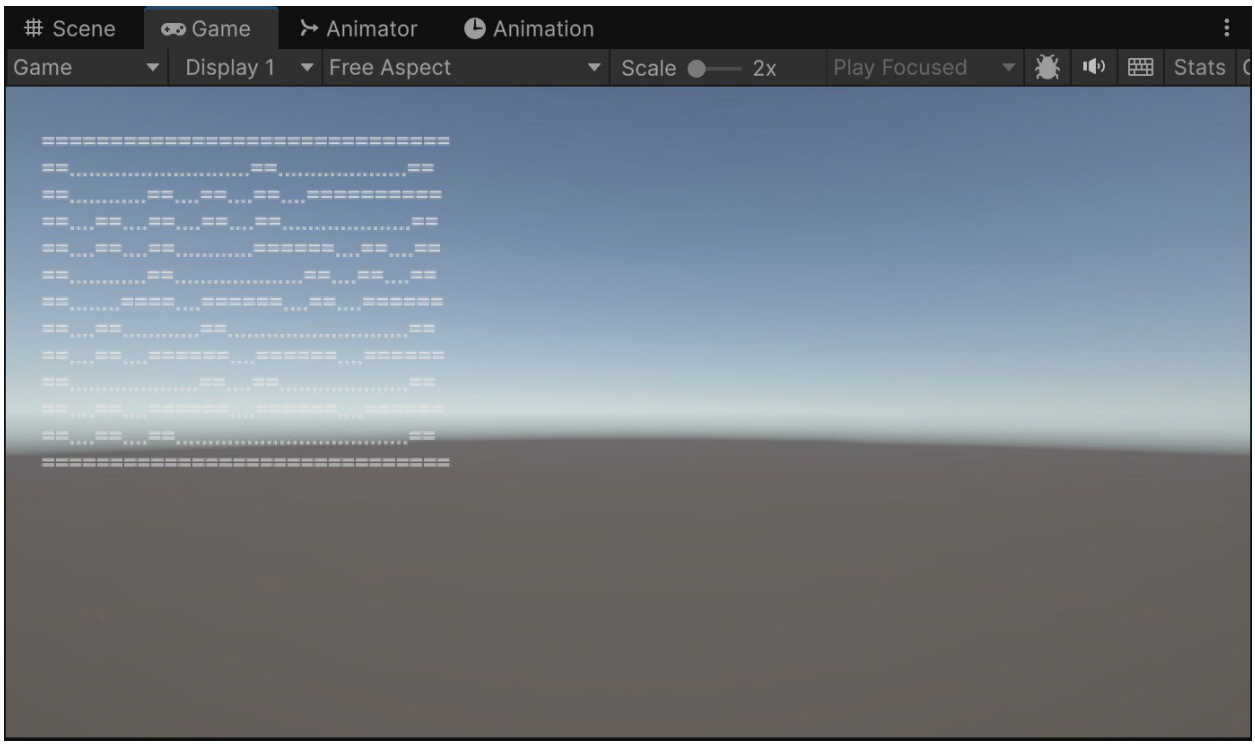

Рисунок 1 – Генерация лабиринта

3.3. Создание генератора меша и отображение тестового четырёхугольника

Был разработан базовый генератор меша:

Создан класс MazeMeshGenerator с параметрами width и height

Настроены материалы (floor-mat, wall-mat, start, treasure) в Inspector

Создан тег Generated для идентификации процедурных объектов

Реализован метод FromData(), создающий единственный четырёхугольник:

Созданы списки вершин, UV-координат и треугольников

Определены 4 вершины для квада размером 1×1

Заданы UV-координаты для текстурирования

Сформированы два треугольника (индексы 0,1,2 и 0,2,3)

Добавлен метод DisplayMaze() в MazeConstructor, который:

Создаёт GameObject с компонентами MeshFilter, MeshCollider, MeshRenderer

Назначает сгенерированный меш и материалы

Устанавливает тег Generated

В результате при запуске отображается затекстурированный четырёхугольник

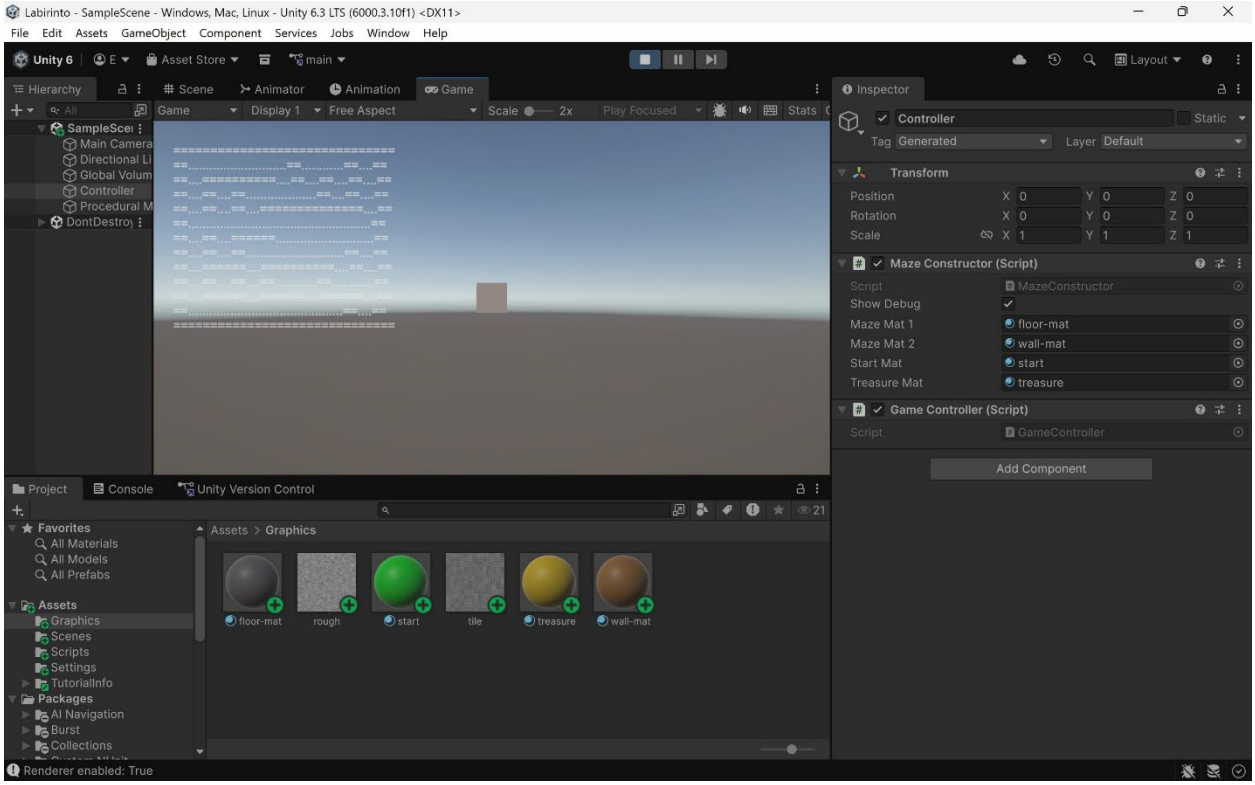

Рисунок 2 – Настройка тестового прямоугольника

3.4. Рефакторинг кода для генерации полноценного 3D-лабиринта

Был полностью переработан метод FromData() для создания целого уровня:

Добавлена поддержка подмешей (subMeshCount = 2) для разделения материалов пола и стен

Созданы отдельные списки треугольников: floorTriangles и wallTriangles

Реализован вспомогательный метод AddQuad() для многократной генерации четырёхугольников:

Использует матрицу преобразований Matrix4x4.TRS() для позиционирования, поворота и масштабирования

Применяет трансформацию через MultiplyPoint3x4() к вершинам

Автоматически рассчитывает индексы вершин для треугольников

Переписан основной цикл генерации в FromData():

Проходит по всем ячейкам 2D-массива данных лабиринта

Для каждой открытой ячейки создаёт пол и потолок с помощью AddQuad()

Проверяет соседние ячейки на наличие стен и создаёт стены только где нужно

Использует разные матрицы преобразований для каждой стороны (вперёд, назад, влево, вправо)

Добавлен вызов RecalculateNormals() для корректного освещения

В результате при запуске генерируется полноценный 3D-лабиринт с полом, потолком и стенами

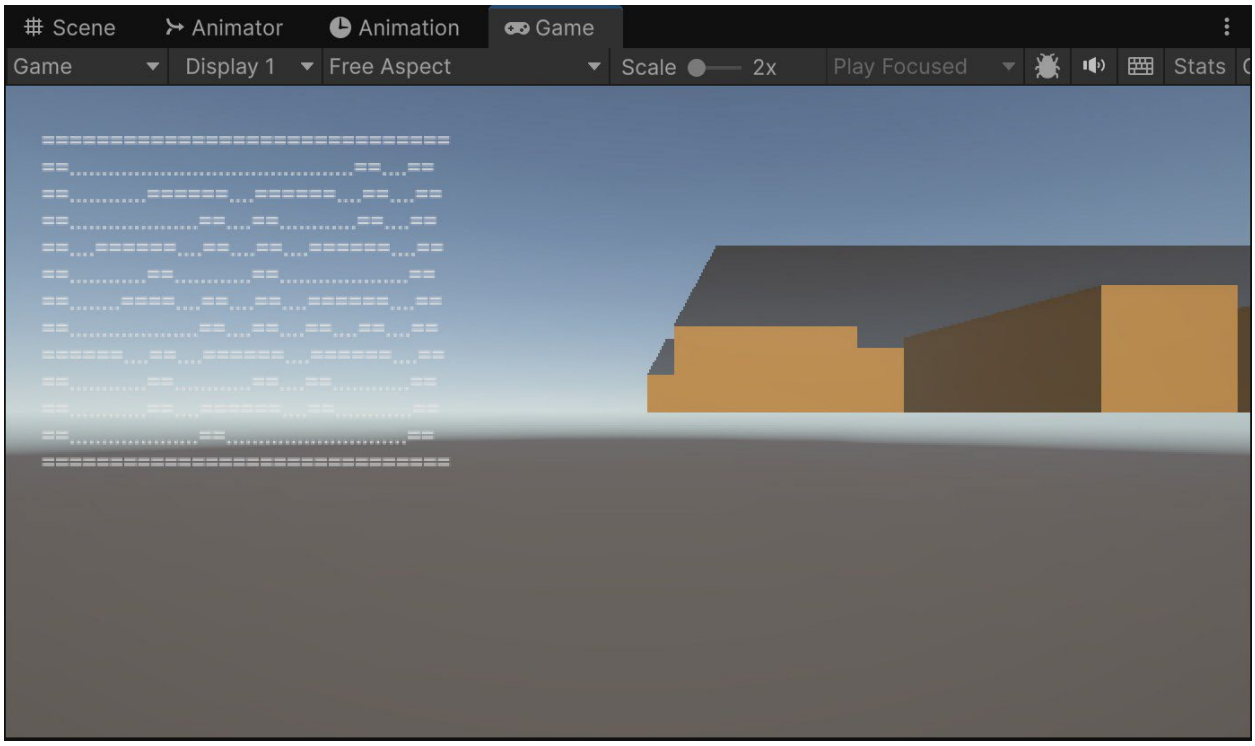

Рисунок 3.1 – Настройка генерации 3D лабиринта

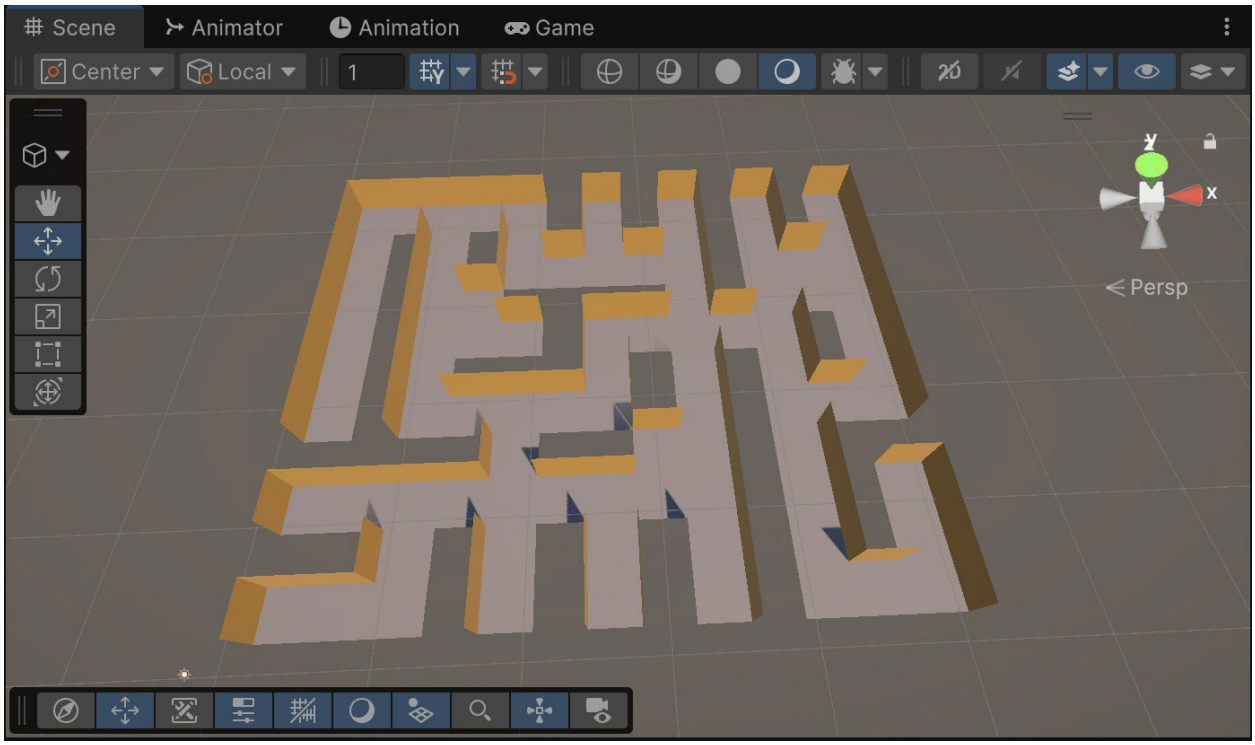

Рисунок 3.2 – Проверка генерации 3D лабиринта в окне сцены

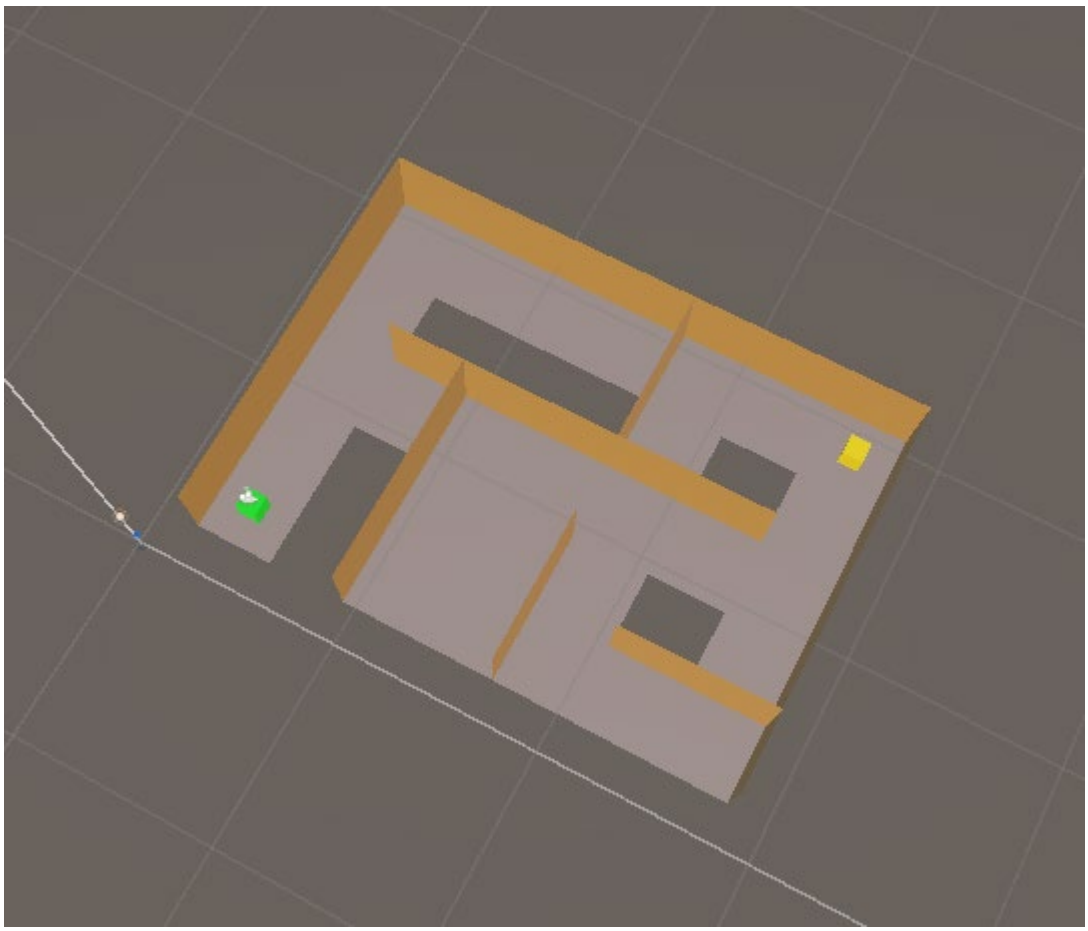

Рисунок 3.3 – Лабиринт 7 на 9

Размер представлен, установленный по-умолчанию: 19, 17.

Также проверена меньшая размерность 7, 9.

3.5. Завершение разработки игры и добавление игровой логики

Была реализована полноценная игра с механикой прохождения уровней:

Добавлены свойства для хранения размеров коридоров (hallWidth, hallHeight) и координат начала/цели

Реализованы методы поиска позиций:

FindStartPosition() – находит первое открытое пространство с начала массива

FindGoalPosition() – находит последнее открытое пространство с конца массива

Созданы методы размещения триггеров:

PlaceStartTrigger() – размещает стартовую зону с коллайдером-триггером

PlaceGoalTrigger() – размещает триггер цели (сундук с сокровищем)

Оба используют компонент TriggerEventRouter из заготовки проекта

Переработан метод GenerateNewMaze() для:

Удаления старого лабиринта (DisposeOldMaze())

Генерации новых данных и поиска стартовой позиции и позиции цели

Создания меша и размещения триггеров с callback-функциями

Реализована игровая логика в GameController:

Метод StartNewGame() – полный перезапуск игры со сбросом очков

Метод StartNewMaze() – переход на следующий уровень с уменьшением лимита времени на 5 секунд

Метод Update() – отслеживание оставшегося времени и завершение уровня при его истечении

Обработчики событий OnGoalTrigger() и OnStartTrigger() - увеличение очков при нахождении сокровища и переход на следующий уровень

Настроена система времени: начальный лимит 80 секунд

Интегрированы UI-элементы (метки времени и очков) с системой UnityEngine.UI

Включён Canvas и назначены все необходимые ссылки в Inspector

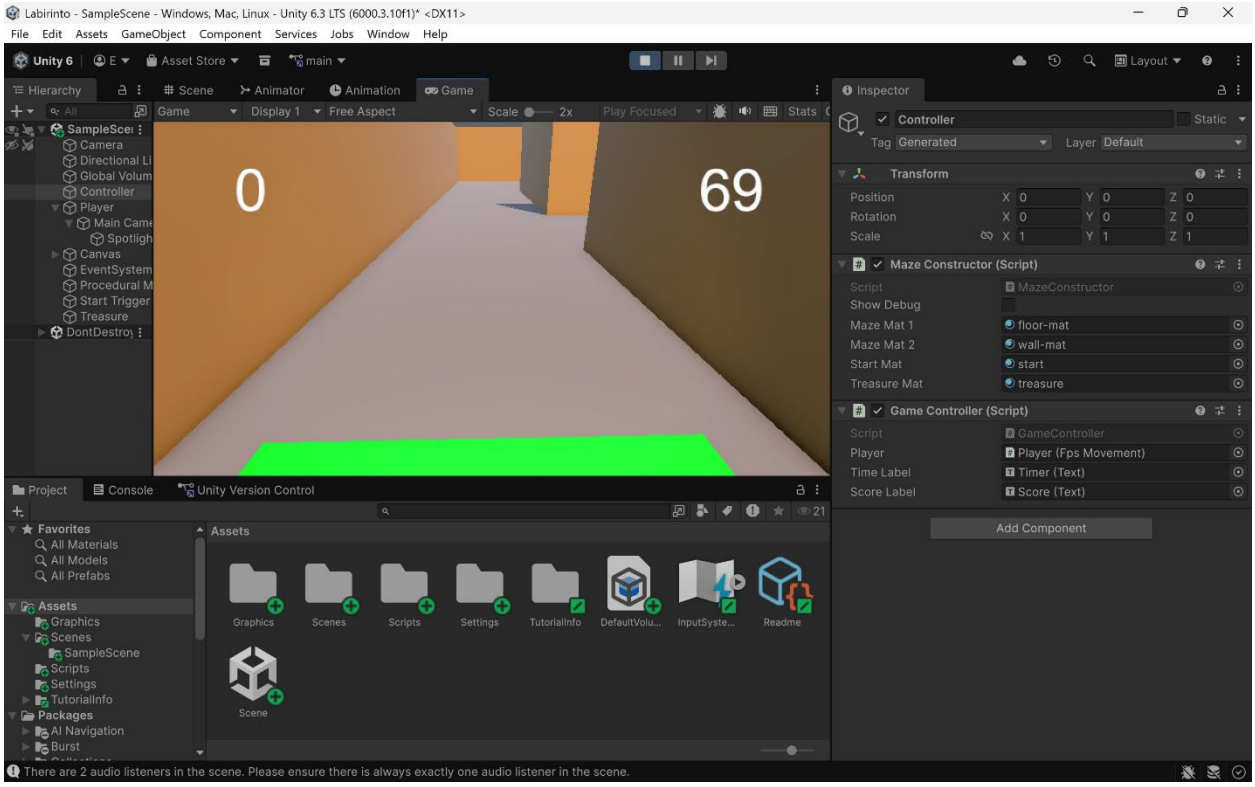

Рисунок 4 – Настройка скриптов управления камерой, тестирование

3.6. Классификация лабиринта

Классификация алгоритма генерации лабиринта:

Измерение: 2D – Используется двумерный массив int[,] data. Лабиринт строится по осям X и Z (высота Y фиксирована).

Гиперразмерность: Нет (границы замкнуты стенами) – Границы массива всегда становятся стенами

Топология: Сетка (Grid) – Данные хранятся в прямоугольной сетке ячеек. Каждая ячейка имеет чётких соседей (верх, низ, лево, право).

Тесселяция: Прямоугольная – Ячейки лабиринта представляют собой квадраты/прямоугольники (в Unity это квады, собранные в сетку).

Маршрутизация: С петлями (Braid / Loopy) – лабиринт неидеальный, содержит циклы. – В уроке сказано: *"лабиринты неидеальны и в них есть петли"*. Алгоритм не гарантирует единственный путь (не является "Perfect Maze"), он создаёт циклы и открытые пространства.

Текстура: Равномерная (случайное распределение стен) – Стены размещаются случайно на основе placementThreshold без смещения в какую-либо сторону (нет вертикального или горизонтального_bias).

Фокус: Добавление стен (Wall-adder) – заполнение пустого пространства стенами по вероятности. – Массив изначально создаётся пустым (нули), а код явно присваивает 1 (стена) границам и случайным ячейкам.

ВЫВОД

В ходе лабораторной работы реализована процедурная генерация лабиринтов в среде Unity на примере игры «Speedy Treasure Thief». Разработана модульная архитектура проекта, включающая контроллер игры, конструктор лабиринта и специализированные генераторы данных и мешей.

Реализованный алгоритм классифицирован следующим образом: измерение – 2D, топология – сетка (Grid), тесселяция – прямоугольная, гиперразмерность – отсутствует, маршрутизация - с петлями (Braid), текстура – равномерная, фокус – добавление стен.

Создан полноценный игровой цикл с системой времени, подсчётом очков и условиями победы. Задание выполнено в полном объёме, функциональность протестирована.

ПРИЛОЖЕНИЕ А

ЛИСТИНГ

GameController:

```
using System;
using UnityEngine;
using UnityEngine.UI;

[RequireComponent(typeof(MazeConstructor))]

public class GameController : MonoBehaviour
{
    //1
    [SerializeField] private FpsMovement player;
    [SerializeField] private Text timeLabel;
    [SerializeField] private Text scoreLabel;

    private MazeConstructor generator;

    //2
    private DateTime startTime;
    private int timeLimit;
    private int reduceLimitBy;

    private int score;
    private bool goalReached;

    //3
    void Start()
    {
        generator = GetComponent<MazeConstructor>();
        StartNewGame();
    }

    //4
    private void StartNewGame()
```

```
{
    timeLimit = 80;
    reduceLimitBy = 5;
    startTime = DateTime.Now;

    score = 0;
    scoreLabel.text = score.ToString();

    StartNewMaze();
}

//5
private void StartNewMaze()
{
    generator.GenerateNewMaze(7, 9, OnStartTrigger, OnGoalTrigger);

    float x = generator.startCol * generator.hallWidth;
    float y = 1;
    float z = generator.startRow * generator.hallWidth;
    player.transform.position = new Vector3(x, y, z);

    goalReached = false;
    player.enabled = true;

    // restart timer
    timeLimit -= reduceLimitBy;
    startTime = DateTime.Now;
}

//6
void Update()
{
    if (!player.enabled)
    {
        return;
```

```
    }

    int timeUsed = (int)(DateTime.Now - startTime).TotalSeconds;
    int timeLeft = timeLimit - timeUsed;

    if (timeLeft > 0)
    {
        timeLabel.text = timeLeft.ToString();
    }
    else
    {
        timeLabel.text = "TIME UP";
        player.enabled = false;

        Invoke("StartNewGame", 4);
    }
}

//7
private void OnGoalTrigger(GameObject trigger, GameObject other)
{
    Debug.Log("Goal!");
    goalReached = true;

    score += 1;
    scoreLabel.text = score.ToString();

    Destroy(trigger);
}

private void OnStartTrigger(GameObject trigger, GameObject other)
{
    if (goalReached)
    {
        Debug.Log("Finish!");
```

```
            player.enabled = false;

            Invoke("StartNewMaze", 4);
        }
    }
}
```

MazeConstructor.cs

```
using UnityEngine;

public class MazeConstructor : MonoBehaviour
{
    //1
    public bool showDebug;

    [SerializeField] private Material mazeMat1;
    [SerializeField] private Material mazeMat2;
    [SerializeField] private Material startMat;
    [SerializeField] private Material treasureMat;

    private MazeDataGenerator dataGenerator;

    private MazeMeshGenerator meshGenerator;

    //2
    public int[,] data
    {
        get; private set;
    }

    public float hallWidth
    {
        get; private set;
    }
    public float hallHeight
    {
```

```
    get; private set;
}

public int startRow
{
    get; private set;
}
public int startCol
{
    get; private set;
}

public int goalRow
{
    get; private set;
}
public int goalCol
{
    get; private set;
}

private void FindStartPosition()
{
    int[,] maze = data;
    int rMax = maze.GetUpperBound(0);
    int cMax = maze.GetUpperBound(1);

    for (int i = 0; i <= rMax; i++)
    {
        for (int j = 0; j <= cMax; j++)
        {
            if (maze[i, j] == 0)
            {
                startRow = i;
                startCol = j;
```

```
                return;
            }
        }
    }
}

private void FindGoalPosition()
{
    int[,] maze = data;
    int rMax = maze.GetUpperBound(0);
    int cMax = maze.GetUpperBound(1);

    // loop top to bottom, right to left
    for (int i = rMax; i >= 0; i--)
    {
        for (int j = cMax; j >= 0; j--)
        {
            if (maze[i, j] == 0)
            {
                goalRow = i;
                goalCol = j;
                return;
            }
        }
    }
}

private void PlaceStartTrigger(TriggerEventHandler callback)
{
    GameObject go = GameObject.CreatePrimitive(PrimitiveType.Cube);
    go.transform.position = new Vector3(startCol * hallWidth, .5f, startRow *
hallWidth);
    go.name = "Start Trigger";
    go.tag = "Generated";
```

```
        go.GetComponent<BoxCollider>().isTrigger = true;
        go.GetComponent<MeshRenderer>().sharedMaterial = startMat;

        TriggerEventRouter tc = go.AddComponent<TriggerEventRouter>();
        tc.callback = callback;
    }

    private void PlaceGoalTrigger(TriggerEventHandler callback)
    {
        GameObject go = GameObject.CreatePrimitive(PrimitiveType.Cube);
        go.transform.position = new Vector3(goalCol * hallWidth, .5f, goalRow *
hallWidth);
        go.name = "Treasure";
        go.tag = "Generated";

        go.GetComponent<BoxCollider>().isTrigger = true;
        go.GetComponent<MeshRenderer>().sharedMaterial = treasureMat;

        TriggerEventRouter tc = go.AddComponent<TriggerEventRouter>();
        tc.callback = callback;
    }


    //3
    void Awake()
    {
        meshGenerator = new MazeMeshGenerator();

        dataGenerator = new MazeDataGenerator();

        // default to walls surrounding a single empty cell
        data = new int[,]
        {
            {1, 1, 1},
            {1, 0, 1},
```

```
        {1, 1, 1}
    };
}

private void DisplayMaze()
{
    GameObject go = new GameObject();
    go.transform.position = Vector3.zero;
    go.name = "Procedural Maze";
    go.tag = "Generated";

    MeshFilter mf = go.AddComponent<MeshFilter>();
    mf.mesh = meshGenerator.FromData(data);

    MeshCollider mc = go.AddComponent<MeshCollider>();
    mc.sharedMesh = mf.mesh;

    MeshRenderer mr = go.AddComponent<MeshRenderer>();
    mr.materials = new Material[2] { mazeMat1, mazeMat2 };

    // 🔍 Отладка
    Debug.Log($"Mesh vertices: {mf.mesh.vertexCount}");
    Debug.Log($"Mesh bounds: {mf.mesh.bounds}");
    Debug.Log($"Renderer enabled: {mr.enabled}");
}

public void GenerateNewMaze(int sizeRows, int sizeCols,
TriggerEventHandler startCallback = null, TriggerEventHandler goalCallback = null)
{
    if (sizeRows % 2 == 0 && sizeCols % 2 == 0)
    {
        Debug.LogError("Odd numbers work better for dungeon size.");
    }
```

```
    data = dataGenerator.FromDimensions(sizeRows, sizeCols);

    FindStartPosition();
    FindGoalPosition();

    // store values used to generate this mesh
    hallWidth = meshGenerator.width;
    hallHeight = meshGenerator.height;

    DisplayMaze();

    PlaceStartTrigger(startCallback);
    PlaceGoalTrigger(goalCallback);
}


void OnGUI()
{
    //1
    if (!showDebug)
    {
        return;
    }

    //2
    int[,] maze = data;
    int rMax = maze.GetUpperBound(0);
    int cMax = maze.GetUpperBound(1);

    string msg = "";

    //3
    for (int i = rMax; i >= 0; i--)
    {
        for (int j = 0; j <= cMax; j++)
```

```
            {
                if (maze[i, j] == 0)
                {
                    msg += "....";
                }
                else
                {
                    msg += "==";
                }
            }
            msg += "\n";
        }

        //4
        GUI.Label(new Rect(20, 20, 500, 500), msg);
    }

}
```

MazeDataGenerator.cs

```
using System.Collections.Generic;
using UnityEngine;

public class MazeDataGenerator
{
    public float placementThreshold;    // chance of empty space

    public MazeDataGenerator()
    {
        placementThreshold = .1f;                           // 1
    }

    public int[,] FromDimensions(int sizeRows, int sizeCols)    // 2
    {
        int[,] maze = new int[sizeRows, sizeCols];
        int rMax = maze.GetUpperBound(0);
```

```
        int cMax = maze.GetUpperBound(1);

        for (int i = 0; i <= rMax; i++)
        {
            for (int j = 0; j <= cMax; j++)
            {
                //1
                if (i == 0 || j == 0 || i == rMax || j == cMax)
                {
                    maze[i, j] = 1;
                }

                //2
                else if (i % 2 == 0 && j % 2 == 0)
                {
                    if (Random.value > placementThreshold)
                    {
                        //3
                        maze[i, j] = 1;

                        int a = Random.value < .5 ? 0 : (Random.value < .5 ? -1 : 1);
                        int b = a != 0 ? 0 : (Random.value < .5 ? -1 : 1);
                        maze[i + a, j + b] = 1;
                    }
                }
            }
        }

        return maze;
    }
}
```

MazeMeshGenerator.cs

```
using System.Collections.Generic;
using UnityEngine;
```

```
public class MazeMeshGenerator
{
    // generator params
    public float width;     // how wide are hallways
    public float height;    // how tall are hallways

    public MazeMeshGenerator()
    {
        width = 3.75f;
        height = 3.5f;
    }

    public Mesh FromData(int[,] data)
    {
        Mesh maze = new Mesh();

        //3
        List<Vector3> newVertices = new List<Vector3>();
        List<Vector2> newUVs = new List<Vector2>();

        maze.subMeshCount = 2;
        List<int> floorTriangles = new List<int>();
        List<int> wallTriangles = new List<int>();

        int rMax = data.GetUpperBound(0);
        int cMax = data.GetUpperBound(1);
        float halfH = height * .5f;

        //4
        for (int i = 0; i <= rMax; i++)
        {
            for (int j = 0; j <= cMax; j++)
            {
                if (data[i, j] != 1)
                {
```

```
// floor
AddQuad(Matrix4x4.TRS(
    new Vector3(j * width, 0, i * width),
    Quaternion.LookRotation(Vector3.up),
    new Vector3(width, width, 1)
), ref newVertices, ref newUVs, ref floorTriangles);

// ceiling
AddQuad(Matrix4x4.TRS(
    new Vector3(j * width, height, i * width),
    Quaternion.LookRotation(Vector3.down),
    new Vector3(width, width, 1)
), ref newVertices, ref newUVs, ref floorTriangles);


// walls on sides next to blocked grid cells

if (i - 1 < 0 || data[i - 1, j] == 1)
{
    AddQuad(Matrix4x4.TRS(
        new Vector3(j * width, halfH, (i - .5f) * width),
        Quaternion.LookRotation(Vector3.forward),
        new Vector3(width, height, 1)
    ), ref newVertices, ref newUVs, ref wallTriangles);
}

if (j + 1 > cMax || data[i, j + 1] == 1)
{
    AddQuad(Matrix4x4.TRS(
        new Vector3((j + .5f) * width, halfH, i * width),
        Quaternion.LookRotation(Vector3.left),
        new Vector3(width, height, 1)
    ), ref newVertices, ref newUVs, ref wallTriangles);
}
```

```
            if (j - 1 < 0 || data[i, j - 1] == 1)
            {
                AddQuad(Matrix4x4.TRS(
                    new Vector3((j - .5f) * width, halfH, i * width),
                    Quaternion.LookRotation(Vector3.right),
                    new Vector3(width, height, 1)
                ), ref newVertices, ref newUVs, ref wallTriangles);
            }

            if (i + 1 > rMax || data[i + 1, j] == 1)
            {
                AddQuad(Matrix4x4.TRS(
                    new Vector3(j * width, halfH, (i + .5f) * width),
                    Quaternion.LookRotation(Vector3.back),
                    new Vector3(width, height, 1)
                ), ref newVertices, ref newUVs, ref wallTriangles);
            }
        }
    }
}

maze.vertices = newVertices.ToArray();
maze.uv = newUVs.ToArray();

maze.SetTriangles(floorTriangles.ToArray(), 0);
maze.SetTriangles(wallTriangles.ToArray(), 1);

//5
maze.RecalculateNormals();

return maze;
}

//1, 2
private void AddQuad(Matrix4x4 matrix, ref List<Vector3> newVertices,
```

```
        ref List<Vector2> newUVs, ref List<int> newTriangles)
    {
        int index = newVertices.Count;

        // corners before transforming
        Vector3 vert1 = new Vector3(-.5f, -.5f, 0);
        Vector3 vert2 = new Vector3(-.5f, .5f, 0);
        Vector3 vert3 = new Vector3(.5f, .5f, 0);
        Vector3 vert4 = new Vector3(.5f, -.5f, 0);

        newVertices.Add(matrix.MultiplyPoint3x4(vert1));
        newVertices.Add(matrix.MultiplyPoint3x4(vert2));
        newVertices.Add(matrix.MultiplyPoint3x4(vert3));
        newVertices.Add(matrix.MultiplyPoint3x4(vert4));

        newUVs.Add(new Vector2(1, 0));
        newUVs.Add(new Vector2(1, 1));
        newUVs.Add(new Vector2(0, 1));
        newUVs.Add(new Vector2(0, 0));

        newTriangles.Add(index + 2);
        newTriangles.Add(index + 1);
        newTriangles.Add(index);

        newTriangles.Add(index + 3);
        newTriangles.Add(index + 2);
        newTriangles.Add(index);
    }

}
```